LEVEL 8

Chapter 9: API Design

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026

Chapter 9: API Design

← *Chapter 8: Microservices* · *Index* · *Next* → *Chapter 10: Security*

9.1 REST vs GraphQL vs gRPC — Choosing, Not Just Defining

You met these in Chapter 1; here's the interview-depth version, focused on the decision.

GraphQL — a query language where the *client* specifies exactly which fields it needs, across potentially many related entities, in a single request. Solves two specific REST pain points: **over-fetching** (a REST endpoint returns a fixed shape, even if the client only needs 2 of 15 fields) and **under-fetching** (a mobile screen needing data from 3 different REST endpoints has to make 3 round trips, or you build a bespoke aggregation endpoint for it). A single GraphQL query can fetch a user, their last 5 orders, and each order's items in one round trip.

Trade-offs: a single flexible endpoint makes response-level HTTP caching much harder (every query can be shaped differently, so you can't cache by URL the way REST naturally allows), it's easier for a client to accidentally construct an expensive query (deeply nested queries can be very costly server-side — needs query complexity limits), and it shifts real complexity into the resolver layer on the backend.

	REST	GraphQL	gRPC
Best for	Public APIs, simple CRUD, cacheable resources	Client-driven data needs, mobile apps with varying screen data needs, aggregating multiple backend sources	Internal service-to-service, streaming, performance-critical paths
Caching	Easy (HTTP caching by URL)	Hard (needs custom solutions)	N/A (not typically cached the same way)
Learning curve for consumers	Low, universally understood	Medium — needs schema knowledge, tooling	Higher — needs generated stubs, protobuf familiarity
Over/under-fetching	Common problem	Solved by design	N/A — you define exact RPC methods

> **Interview question:** "Your mobile app team complains they need 4 API calls to render one screen. What would you consider?"
> **Ideal senior answer:** "That's a classic under-fetching symptom. Before reaching for GraphQL — which is a real architectural commitment — I'd first consider a simpler **Backend-for-Frontend (BFF)** pattern: one aggregation endpoint tailored to that screen's exact needs, composing calls to the underlying services server-side. If this pattern repeats across many different screens with genuinely different, unpredictable data shapes, that's when GraphQL's generality starts earning its complexity cost — I wouldn't reach for it for one screen's problem."

9.2 API Versioning

Why it's unavoidable: once an API has external consumers (mobile apps you don't control the release cadence of, partner integrations), you cannot force everyone to upgrade the moment you change something — old clients keep calling the old contract for a long time.

Strategy	Example	Trade-off
URI versioning	`/v1/orders`, `/v2/orders`	Most explicit and cache-friendly; clutters the URL space; most common in practice
Header versioning	`Accept: application/vnd.myapi.v2+json`	Cleaner URLs; less discoverable, harder to test with a browser/curl casually
Query param versioning	`/orders?version=2`	Simple, but easy to forget/omit, less semantically clean

The bigger rule: prefer **additive, backward-compatible changes** (new optional fields, new endpoints) over versioning at all, wherever possible — versioning is the tool for genuine breaking changes, not the default way you evolve an API.

9.3 Pagination

Offset pagination: `GET /orders?offset=100&limit=20` — simple, supports "jump to page 5," but has two real problems at scale: performance (the database still has to scan/skip past the first 100 rows even though it discards them — `OFFSET` gets slower as it grows) and **consistency** (if rows are inserted/deleted between page requests, results can shift — a user can see the same item twice or skip one entirely).

Cursor pagination: `GET /orders?after=order_id_456&limit=20` — the cursor (typically an opaque encoded pointer, often based on a unique, sortable column like `id` or `created_at`) tells the database exactly where to resume, which is a direct indexed lookup (`WHERE id > 456 LIMIT 20`), not a skip-and-discard scan. It's also stable against concurrent inserts/deletes in a way offset pagination isn't. The cost: you lose the ability to jump directly to "page 5" — you can only page forward/backward from a known cursor.

> **Interview question:** "Why does cursor pagination scale better than offset pagination for an infinite-scroll feed?"
> **Ideal senior answer:** "Two reasons. Performance: `OFFSET 100000` still requires the database to walk through and discard 100,000 rows before returning your page, which gets linearly worse as users scroll deeper — a cursor-based `WHERE id > lastSeenId` is a direct indexed seek regardless of how deep you are. Correctness: a feed is constantly being written to by other users; offset pagination can show you duplicate or skipped items as new posts shift everyone's offsets, while a cursor anchored to a specific item's position doesn't have that problem. Feeds are exactly the use case cursor pagination was built for — that's why Twitter, Instagram, and Facebook all use it, not offset pagination."

Filtering and sorting: standard query parameters (`?status=delivered&sort=-created_at`), but at scale, make sure every filterable/sortable field is actually indexed — an unindexed filter on a large table is a silent full-table-scan waiting to happen in production.

9.4 Idempotency Keys and Retry-Safe APIs

Covered in depth in Chapter 6.4 — the API design summary: any endpoint that has a side effect that shouldn't happen twice (payments, order creation) should accept an `Idempotency-Key` header, store the key with its result, and return the stored result on a repeat key instead of re-executing the side effect. `PUT` (full replace) is naturally idempotent by HTTP semantics; `POST` (create) is not, by default, which is exactly why idempotency keys matter most for POST endpoints with side effects.

9.5 Rate Limiting, Authentication, Authorization, Error Handling

Rate limiting at the API layer (Chapter 13 has the algorithms) — typically enforced per API key/user, returning `429 Too Many Requests` with a `Retry-After` header telling the client when it's safe to try again.

Authentication/authorization at the API layer — validate identity (API key, JWT, OAuth token) before routing to business logic, typically at the gateway (Chapter 4) so individual services don't reimplement it.

Good error handling: return structured, consistent error bodies (`{"error": {"code": "INSUFFICIENT_FUNDS", "message": "...", "request_id": "..."}}`) with the right HTTP status code — not `200 OK` with an error buried in the body (a surprisingly common anti-pattern that breaks client-side error handling, caching, and monitoring, since `200` responses often get treated as automatically successful by infrastructure/monitoring). Include a `request_id` /trace ID in every error response so a user-reported issue can be traced directly to server-side logs.

9.6 Good vs. Bad API Design — Side by Side

Bad:

```
GET /getUserOrders?uid=123
POST /createNewOrderForUser
GET /order_delete?id=456
```

What's wrong: inconsistent naming/casing, verbs baked into the URL instead of using HTTP methods, no resource hierarchy, `GET` used to delete something (breaks caching/prefetching assumptions — a `GET` should never have a side effect), inconsistent verb style across endpoints.

Better:

```
GET    /users/123/orders
POST   /orders
DELETE /orders/456
```

What's better: resource-oriented URLs, HTTP methods carry the verb, consistent, predictable structure a client can guess.

Excellent (production-grade):

```
GET    /v1/users/123/orders?status=delivered&after=order_789&limit=20
POST   /v1/orders                      (requires Idempotency-Key header)
PATCH  /v1/orders/456                  (partial update, not full replace)
DELETE /v1/orders/456                  (soft-delete in practice — returns 204)

Response headers include: X-RateLimit-Remaining, X-Request-Id
Error body: {"error": {"code": "ORDER_NOT_FOUND", "message": "...", "request_id": "..."}}
```

What makes this excellent: versioned, cursor-paginated, explicit idempotency handling for the unsafe write, `PATCH` used correctly for partial updates, consistent structured errors, and observability (request ID) baked into every response — this is the level of detail that separates a mid-level API design answer from a senior one.

Chapter 9 Interview Drill

1. When would you reach for GraphQL over a REST endpoint, and what's the simpler alternative to try first?
2. Explain precisely why cursor pagination outperforms offset pagination for a large, actively-written-to feed.
3. Walk through how an idempotency key prevents a duplicate charge on a `POST /payments` retry.
4. Critique this endpoint: `GET /deleteUser?id=42` — list every problem.
5. Design the endpoints for a "cancel order" feature, including method, path, and idempotency handling.

Next → Chapter 10: Security — authentication, authorization, encryption, and the security surface of a production system.